

CvSU Scheduling System — GA Architecture Deep Dive

Part 1: The Building Blocks

The Gene (Pinakamaliit na unit)

Ang isang Gene ay kumakatawan sa isang scheduled class — isa lang na meeting ng isang subject sa isang section. May 11 fields ito:

Field	Meaning
course_id	Aling subject
section_id	Para sa aling section
faculty_id	Sino ang magtuturo
room_id	Saang kwarto
day_idx	Alin sa allowed days (0=Monday, 1=Tuesday, etc.)
start_idx	Time slot na simula (bawat slot = 30 min, kaya slot 0 = 7:00 AM)
duration_slots	Ilang slots ang haba (e.g., 3-unit lec = 6 slots = 3 hours)
end_idx	start_idx + duration_slots
gene_type	Lec, Lab, o Async

Field	Meaning
is_fixed	Kung True, hindi pwedeng baguhin (pre-assigned)
bitmask	Bitwise pattern ng occupied slots para sa mabilis na overlap detection

Halimbawa: Math 101, Section A, Hapon, Martes, Room 201, 10:00–11:30 AM = isang gene.

Ang bitmask ang trick para sa speed. Sa halip na i-loop ang bawat slot ng overlap, ginagamit nito bitwise AND operation — nagiging single CPU instruction ang pagcheck ng conflict.

The Chromosome (Isang buong schedule)

Ang isang Chromosome = listahan ng lahat ng genes (isang gene per class meeting). Kung may 50 courses × average 2 genes bawat isa × 10 sections = ~1000 genes ang isang chromosome.

May 9 properties ang chromosome:

Property	Meaning
genes	List ng lahat ng Gene objects
fitness	Combined penalty score (mas mababa = mas maganda)
hard_conflicts	Bilang ng HC violations
soft_score	Combined soft constraint penalty

Property	Meaning
conflicting_indices	Indexes ng genes na may conflict (para sa targeted repair)
rank	NSGA-II Pareto rank (unused sa final version — pinalitan ng simple sort)
crowding_distance	NSGA-II diversity measure (nakalaan pa rin pero hindi aktibong ginagamit)
sc1_violations	Bilang ng SC-I violations (displayed sa UI)
sc2_violations	Bilang ng SC-II violations (displayed sa UI)

The Population (Lahat ng candidate schedules)

Ang Population = collection ng maraming chromosomes na sinasabay-sabay na ina-evolve.

Dynamic na ang size: $\max(40, \min(150, n_{\text{genes}} \times 0.35))$

Ibig sabihin:

- Minimum: 40 chromosomes kahit konti ang courses
- Maximum: 150 chromosomes kahit malaki ang schedule
- Middle: ~35% ng total gene count

Halimbawa: kung ~250 genes ang total, $250 \times 0.35 = 87 \rightarrow$ population size = 87.

Bakit dynamic? Mas maraming courses = mas malaking search space = kailangan ng mas maraming diversity.

Part 2: Initialization — Paano nagmumula ang population

Option A: Warm Start (kung may existing schedule)

Ginagamit ito kung may naka-save na schedule na mula sa nakaraang generation.

- 1. Seed chromosome — i-convert ang saved schedule records sa chromosome. Kung may HC, i-repair agad (3 passes).**
- 2. 60% variants — kopyahin ang seed chromosome pero perturb ng 5% hanggang 20% ng genes (graduated: unang copies 5% change, huling copies 20% change). Preserves the good structure ng seed.**
- 3. 40% fresh — ganap na random na bagong schedules. Para may bagong ideas kahit mag-start sa magandang seed.**

Option B: Fresh Start (walang existing schedule)

Bawat chromosome ay ginawa ng create_genome():

- Para sa bawat section × course combination, gumawa ng Lec/Lab/Async genes**
- Bawat gene ay random na ina-assign ng room, day, at time slot**
- Ginagamit ang _smart_start() na nagpi-prefer ng 7AM–12PM slots (para natural na distribution)**

- Ginagamit ang `_best_room()` na nagpi-pick ng room na angkop sa capacity (70% capacity-fit, 30% warmth bonus)
-

Part 3: Fitness Evaluation — Paano sinusukat ang kalidad

Ito ang pinakamahalagang function sa buong GA. Binibigyan ng score ang bawat chromosome — mas mababa ang score, mas maganda.

Dalawang Mode

Hard-only mode (hard phase): Tinitingnan lang ang HC violations. Mas mabilis kasi hindi na kailangang i-check ang soft constraints. Ginagamit habang $HC > 0$.

Full mode (soft phase): Tinitingnan ang lahat — $HC + SC-I + SC-II$. Ginagamit pagkatapos maabot ng $HC = 0$.

Occupation Sets (ang heart ng HC detection)

Bago mag-loop sa genes, gumagawa ng tatlong bitmask dictionaries:

- `occ_room[(room_id, day_idx)]` → bitmask ng lahat ng slots occupied ng room sa day na iyon
- `occ_fac[(faculty_id, day_idx)]` → bitmask ng slots occupied ng faculty
- `occ_sec[(section_id, day_idx)]` → bitmask ng slots occupied ng section

Para mag-detect ng overlap: `occ_room[(room, day)] & gene.bitmask != 0` → may conflict.

Part 4: Constraint Coverage Map

26 Hard Constraints

Code	Name	Saan checked
HC-01	LOCKED_SCHEDULES	Structural (is_fixed genes hindi binabago)
HC-02	MINOR_SUBJECT_GAP	Checker only (needs full room grid scan)
HC-03	GLOBAL_DAY_RESTRICTION	GA fitness: per-gene day_idx validity check
HC-04	LEC_BEFORE_LAB	GA repair: Lab min_day >= Lec day
HC-05	STRICT_LEC_ALLOC	Structural (create_genome guarantees 1 gene per type)
HC-06	STRICT_LAB_ALLOC	Structural
HC-07	STRICT_ASYNC_ALLOC	Structural

Code	Name	Saan checked
HC-08	STRICT_LEC_DURATION	GA fitness: gene.duration_slots vs _expected_duration
HC-09	STRICT_LAB_DURATION	GA fitness
HC-10	STRICT_ASYNC_LEC_DUR	GA fitness
HC-11	STRICT_ASYNC_LAB_DUR	GA fitness
HC-12	ROOM_CONFLICT	GA fitness: occ_room bitmask overlap
HC-13	FACULTY_CONFLICT	GA fitness: occ_fac bitmask overlap
HC-14	SECTION_DAY_RESTRICTIONS	GA fitness: per-gene day_idx check vs section.allowed_days
HC-15	SECTION_CONFLICT	GA fitness: occ_sec bitmask overlap
HC-16	OPERATING_HOURS	GA fitness: end_idx <= total_slots
HC-	FACULTY_DAILY_AVAILABILITY	GA fitness: occ_fac day-

Code	Name	Saan checked
17		level check
HC-18	FACULTY_AVAILABILITY	GA fitness: day_idx not in _fac_avail_days
HC-19	ROOM_AVAILABILITY	GA fitness: room status check
HC-20	SINGLE_ROOM	N/A (1 gene = 1 room by design)
HC-21	ROOM_SUITABILITY	GA fitness: Lab gene sa Computer Lab room
HC-22	ROOM_CAPACITY	GA fitness: room capacity vs section students
HC-23	PRE_ASSIGNMENT	GA init (pre-assigned genes set as is_fixed)
HC-24	HOURLY_ALIGNMENT	Structural (slots are 30-min aligned by definition)
HC-25	COMPLETE_COURSE_SCHEDULING	Checker only (can't repair missing genes mid-evolution)
HC-	PREASSIGNMENT_EXCLUSIVITY	Structural (fixed genes

Code	Name	Saan checked
26		reserved at init)

GA Coverage: 23/26 hard constraints actively penalized (HC-02, HC-25 = checker only; HC-01/05/06/07/20/24/26 = structurally guaranteed)

8 Soft Constraints Type I (weight × 100 penalty)

Code	Name	In GA?
SC-I-01	LUNCH_BREAK	Yes — 12:00–13:00 slot penalty
SC-I-02	MAX_CONSECUTIVE_STUDENT	Yes — per section day
SC-I-03	MAX_CONSECUTIVE_FACULTY	Yes — per faculty day
SC-I-04	LEC_LAB_PROXIMITY	Yes — Lec/Lab same day or adjacent days
SC-I-05	LEC_LAB_WEEKLY_DIST	Yes — spread across week
SC-I-06	LEC_LAB_SEQUENCE	Yes — Lab after Lec on same/later day
SC-I-	NO_ISOLATED_LECTURES	Yes — no lone 1-gene days

Code	Name	In GA?
07		
SC-I-08	MIN_DAILY_SECTION_LOAD	Yes — section needs minimum load per day

SC-I Coverage: 8/8

3 Soft Constraints Type II (weight × 10 penalty)

Code	Name	In GA?
SC-II-01	EVENING_AVOIDANCE	Yes — start_idx >= eve_slot = -10 per gene
SC-II-02	PE_MORNING_PLACEMENT	Yes — PE/FITT should be morning
SC-II-03	ASYNC_STRATEGIC_PLACEMENT	Yes — Async should be Friday/late week

SC-II Coverage: 3/3

Total: 36/37 constraints in GA (HC-02 excluded)

Part 5: Selection — Sino ang susunod sa susunod na generation

Ginagamit ang lexicographic (HC, soft_score) sort:

1. Pag-sort ng lahat ng chromosomes: pangunahin sa `hard_conflicts` (mas mababa = mas maganda), secondary sa `soft_score` (mas mababa = mas maganda)
2. Pinipili ang tournament selection (`_hard_phase_select`): kumukuha ng 3 random chromosomes, pinagsasabay ang pinakamaganda

Hindi ginagamit ang NSGA-II $O(N^2)$ dominance sort — napalitan ng simple $O(N \log N)$ sort dahil $\sim 100\times$ mas mabilis at nagbibigay ng parehong resulta sa single-objective-like na setup natin.

Part 6: Crossover — Paano nagpi-produce ng bagong offspring

Greedy Conflict-Aware Crossover — hindi ito standard 1-point o 2-point crossover.

Ganito ang proseso:

1. I-copy muna ang lahat ng fixed genes (`is_fixed = True`) — hindi binabago
2. I-shuffle ang order ng non-fixed genes (random order para walang bias)
3. Para sa bawat gene position sa random order:
 - Subukan ang Parent 1's version — kung walang conflict sa mga naka-place na genes, gamitin ito
 - Kung may conflict, subukan ang Parent 2's version — kung walang conflict, gamitin ito

- Kung pareho may conflict, exhaustive scan ng lahat ng rooms × days × times para mahanap ang clean slot

4. Pagkatapos ma-place ang gene, i-update ang occupation sets para sa susunod na genes

Bakit ito mas maganda kaysa standard crossover?

Standard block crossover: kukuha ng buong section ng genes mula sa isang parent — pero ang genes ng ibang section maaaring mag-overlap. Ang result: maraming bagong conflicts na kailangang i-repair.

Ang greedy version: bawat gene ay ine-place na parang nagdepende sa mga naka-place na. Result: madalas na HC=0 ang bata kahit parehong magulang ay may HC conflicts, kasi ina-avoid ng placement ang known conflicts.

Part 7: Mutation — Paano nag-eexplore

Proportional Mutation — hindi random ang binabago:

Kung may HC (hard phase):

- Lahat ng genes sa conflicting_indices ay binabago
- Ginagamit ang randomize_gene_fast(): 60 attempts per gene
 - Attempts 1–35: morning/afternoon preference, smart room selection

- Attempts 36–50: allow evening (para sa dense schedules)
- Attempts 51–60: fully random (last resort)
- Bawat 3rd attempt: try different faculty (para makaiwas sa faculty conflicts)

Kung walang HC (soft phase):

- 40% chance: target genes na may soft violations (matalinong mutation)
- 15% chance: random gene (para may exploration kahit walang violations)

Adaptive Mutation Rate:

- Hard phase: nagsisimula sa 20%, tumaas hanggang 80% habang tumatagal ang stagnation
- Soft phase: nagsisimula sa 50% (aggressive agad para ma-explore ang soft landscape)

Part 8: Repair — Kung hindi gumagana ang mutation

_hard_conflict_repair — ginagamit ito after crossover at para sa LNS offspring.

Inspired ng DSatur Graph Coloring algorithm:

1. Build conflict graph — sino sa mga conflicting genes ang nagko-conflict sa isa't isa?

2. Compute degree — ilan ang conflicting neighbors ng bawat conflicting gene?
3. Sort by degree (highest first) — ang pinakamaraming conflict ang unang iaayos
4. Build clean occupation sets — mula sa lahat ng non-conflicting genes
5. Place each conflicting gene — sa clean space lang, hindi sa conflicting genes

Bakit pinaka-conflicted ang una? Kasi ang pinaka-constrained na gene ay mas mahirap i-place later kapag marami nang naka-occupy na slots. I-resolve muna, para may flexibility pa ang mga susunod.

Part 9: LNS (Large Neighbourhood Search)

Ito ay hard phase only — 15 offspring bawat generation:

1. Kopyahin ang best chromosome
2. I-repair lang ang mga genes sa conflicting_indices (e.g., 14 out of 232 genes = 6%)
3. Mas mabilis kaysa full crossover dahil karamihan sa genes ay hindi hinawakan

Ito ang targeted grenade — ang crossover ay nagbibigay ng bagong ideas, ang LNS ay nagsisigurado na ang best solution ay naayos.

Part 10: Stagnation Handling — Kung hindi na gumagalaw

Dalawang independent counters:

Mild Stagnation (15 generations walang improvement):

- **Palitan ang 25% ng population (worst chromosomes)**
- **70% chance: copy ng elite + repair**
- **30% chance: fresh random genome**
- **stagnation_counter reset, pero severe_counter patuloy**

Severe Stagnation (40 generations walang improvement):

- **Pag-reset ng halos buong population**
- **Panatilihin ang 3 best elites**
- **I-repair ang 5 elites (bagong variants)**
- **Punan ng fresh random genomes**
- **Parehong counters reset**

Diversity Injection (checked every 10 generations):

- **Kung $\geq 85\%$ ng population ay magkaparehong (HC, soft_score) fingerprint → premature convergence**
- **20-generation cooldown para hindi masyadong madalas**
- **Split: 30% keep elites / 30% perturbed elites (40% perturbation) / 40% fresh**

Part 11: Two-Phase Termination

Phase 1 — Hard Phase:

- **Mag-evolve hanggang $HC = 0$ o 20 minutes na ang lumipas**
- **Fitness evaluation: hard-only (fast)**
- **Focus: resolving room/faculty/section conflicts**

Phase 2 — Soft Phase (nagsisimula pagkatapos $HC = 0$):

- **I-re-evaluate ang buong population with full fitness (including soft constraints)**
- **Mag-evolve para ma-minimize ang soft violations**
- **5 minutes lang ang ibibigay, o hanggang 400 generations mula sa $HC=0$**
- **O hanggang $fitness == 0$ (perfect schedule — lahat ng soft constraints satisfied)**

Part 12: Bakit $HC=2$ sa GA pero $HC=21$ sa Constraint Checker?

Ito ang pinakamahalagang tanong. Ang sagot:

19 violations na nakikita ng Checker pero hindi ng GA:

- **$HC=25$ (Complete Course Scheduling) — missing section-course pairs ay hindi ma-detect ng GA kasi walang gene index para i-add sa `conflicting_indices`. Checker lang ang nakakaalam.**

- **HC-08/09/10/11 (Duration constraints)** — nakikita ng GA sa fitness pero ang genes na galing sa warm-start ay may inferred gene_type (mula sa room type), hindi exact. Pag naka-save sa DB, ang duration ay validated ng checker gamit ang iba pang logic.
- **HC-01 (Locked Schedules)** — structurally enforced ng GA pero ang checker ay nag-check ng exact time string format. Naayos na ito (zero-padding ng hours).

Summary: Mga Numero ng System

Parameter	Value
Population size	40–150 (dynamic, ~35% ng gene count)
Offspring per gen (hard)	20 crossover + 15 LNS = 35 total
Offspring per gen (soft)	20 crossover
Max generations	2,000
Hard phase limit	20 minutes
Soft phase limit	5 minutes after HC=0
Stagnation mild	15 generations
Stagnation severe	40 generations
Diversity inject	≥85% similarity, 20-gen cooldown

Parameter	Value
HC penalty	1,000,000 per violation
SC-I penalty	weight $\times$ 100
SC-II penalty	weight $\times$ 10
Constraint coverage	36/37 (97.3%)
Hard phase selection	Tournament (k=3)
Sort method	Lexicographic (HC, soft_score)

May tanong ka sa kahit anong part? O gusto mo nang mag-dive sa specific na problem — yung HC=2 vs HC=21 gap o yung 107 evening avoidance violations?